



Large Language Models (LLMs) have multiple categories of hyperparameters.

Training hyperparameters

Architecture hyperparameters

Inference (generation) hyperparameters

Fine-Tuning Specific Hyperparameters (PEFT)

Training Hyperparameters

These control how the model learns during training or fine-tuning.

◆ Learning Rate

- Controls the step size during gradient updates.
- Too high → unstable training
- Too low → slow convergence
- Use: Most important hyperparameter; often tuned with schedulers.

◆ Batch Size

- Number of training samples processed in one step.
- Large batch → stable gradients but high memory usage.
- Small batch → noisy gradients but better generalization sometimes.

◆ Number of Epochs

- How many times the model sees the full dataset.
- Too many → overfitting
- Too few → underfitting



◆ **Optimizer**

- Algorithm used to update weights (e.g., AdamW).
- Controls convergence behavior and stability.

◆ **Weight Decay**

- L2 regularization term.
- Prevents overfitting by penalizing large weights.

◆ **Warmup Steps**

- Gradually increases learning rate at start.
- Prevents early training instability (very important for transformers).

◆ **Gradient Clipping**

- Prevents exploding gradients by capping max gradient value.



Architecture Hyperparameters

These define the structure of the LLM.

To understand these, let's reference a transformer-based LLM like:

OpenAI's GPT models

Google DeepMind's Gemini

Meta's LLaMA

◆ Number of Layers (Depth)

- Number of transformer blocks.
- More layers → more expressive power.
- Example: 12 layers vs 96 layers.

◆ Hidden Size (Embedding Dimension)

- Size of token representation vector.
- Larger dimension → richer representation but more compute.

◆ Number of Attention Heads

- Parallel attention mechanisms per layer.
- Helps model attend to different relationships simultaneously.

◆ Feedforward Network Size (FFN dimension)

- Size of intermediate dense layer inside transformer.
- Usually 4x hidden size.



◆ Context Window (Sequence Length)

- Maximum tokens the model can process.
- Larger context → better long-document reasoning.
- Tradeoff: Memory & compute cost.

◆ Vocabulary Size

- Number of unique tokens.
- Larger vocabulary → fewer token splits but bigger embedding matrix.

◆ Dropout Rate

- Randomly drops neurons during training.
- Helps prevent overfitting.



Inference / Generation Hyperparameters

These control how the model generates text.

These are extremely important in production LLM systems.

◆ Temperature

- Controls randomness.
- Low (0–0.3) → deterministic, factual
- High (0.8–1.5) → creative, diverse

Use:

- Chatbot → 0.7
- Code generation → 0.1–0.3

◆ Top-k Sampling

- Model selects from top k probable tokens.
- $k=50$ → choose from top 50 tokens.

Controls diversity but limits extreme randomness.

◆ Top-p (Nucleus Sampling)

- Selects tokens whose cumulative probability $\geq p$.
- $p=0.9$ → dynamic cutoff.

More adaptive than top-k.

◆ Max Tokens

- Maximum tokens generated.

Prevents long runaway responses



◆ **Stop Sequences**

- Stops generation when a specific token/string appears.
- Used in structured outputs.

◆ **Repetition Penalty**

- Penalizes repeated tokens.
- Reduces looping outputs.

◆ **Presence & Frequency Penalty**

- Presence penalty → discourages repeating concepts
- Frequency penalty → discourages repeating exact words



Fine-Tuning Specific Hyperparameters (PEFT)

When using techniques like LoRA:

◆ LoRA Rank (r)

- Size of low-rank matrices.
- Higher $r \rightarrow$ better adaptation but more parameters.

◆ Alpha (Scaling Factor)

- Controls impact of LoRA updates.

◆ Target Modules

- Which layers to adapt (e.g., query/key projections).



Ketan Sagare

Follow For More!